

西安电子科技大学

A 级达标线上测试报告



学院 计算机科学与技术 专业 计算机科学与技术

学号 23009201247

姓名 郑墨涵

手机 18732105138 完成日期 2026-09-25

成绩

题目名称： 距离测控仿真系统

一、题目要求

软件环境：Proteus 8.17 SP2 及以上仿真软件、Arduino IDE、虚拟串口驱动软件 Virtual Serial Port Driver (VSPD)。

实现功能：使用 Arduino UNO 微控制器，搭建一个 PC 上位机远程距离检测控制系统。Arduino UNO (ATmega328P) 通过串行接口组件 (COMPIM) 与上位机 PC 进行双向通信：PC 上位机软件向 Arduino UNO 发送学生自己的学号，Arduino UNO 收到后向 PC 机发送当前的距离值，并在 LCD 上显示学生的学号、当前的距离值；PC 上位机软件显示接收到的距离值。

控制功能：Arduino UNO 控制驱动直流电机，当距离值高于距离阈值 (30+学号末位数) cm 时，启动直流电机转动；当距离值低于 (含等于) 距离阈值时，直流电机停止转动。同时，实时距离值在 LCD 和 PC 上位机软件显示。

显示要求：LCD 第一行显示 ID: 学号，第二行显示 DIST: 距离值。

上位机要求：必须自行编写 PC 上位机软件，实现 PC 与 Arduino 的双向数据传输及管理控制，编程语言不限。上位机软件 GUI 界面 Title 必须显示学生自己的学号和姓名；必须有发送窗口显示发送的学号；必须有接收窗口显示接收到的距离值；GUI 界面上需要有串口打开关闭功能。

二、仿真结果展示

1. 距离阈值计算过程、计算结果

本人学号为 23009201247，学号末位数为 7。

距离阈值 = $30 + \text{学号末位数} = 30 + 7 = 37 \text{ cm}$ 。

控制规则：距离值 $> 37 \text{ cm}$ 时启动直流电机转动；距离值 $\leq 37 \text{ cm}$ 时直流电机停止。

Arduino 程序收到上位机发来的学号后，从学号字符串末位自动按上式计算阈值 ($\text{threshold} = 30 + \text{学号末位}$)。

2. 距离高于距离阈值时的截图 (50cm > 37cm, 电机转动)

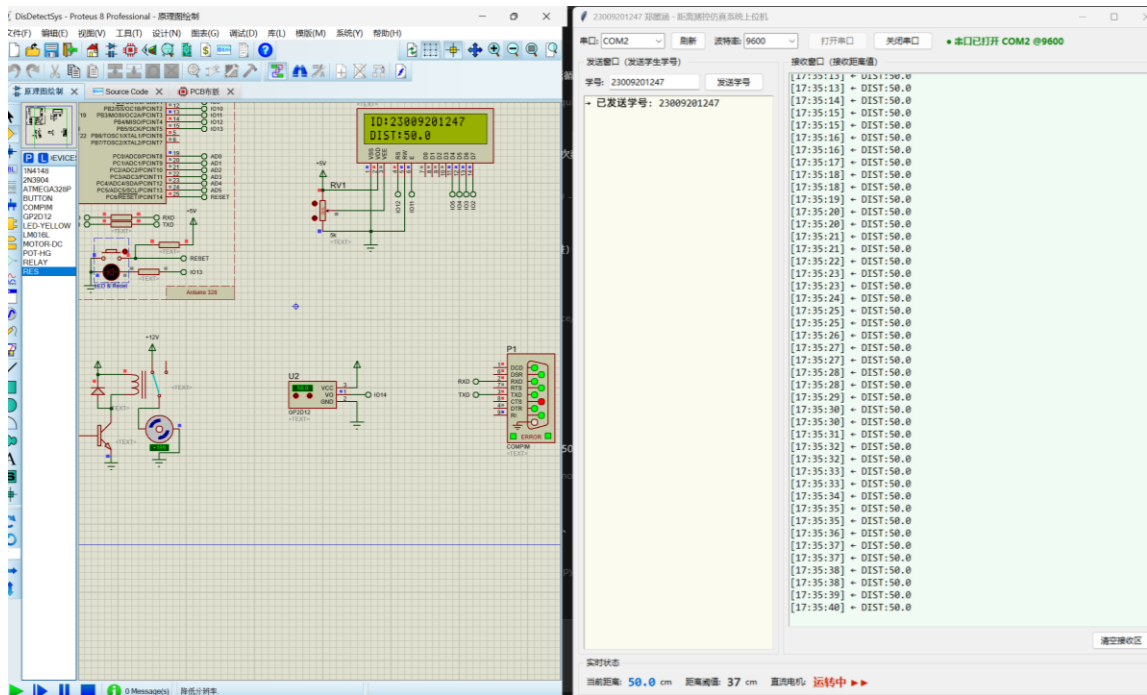


图 1 距离 50.0cm 高于阈值 37cm 时系统全景（电机转动）

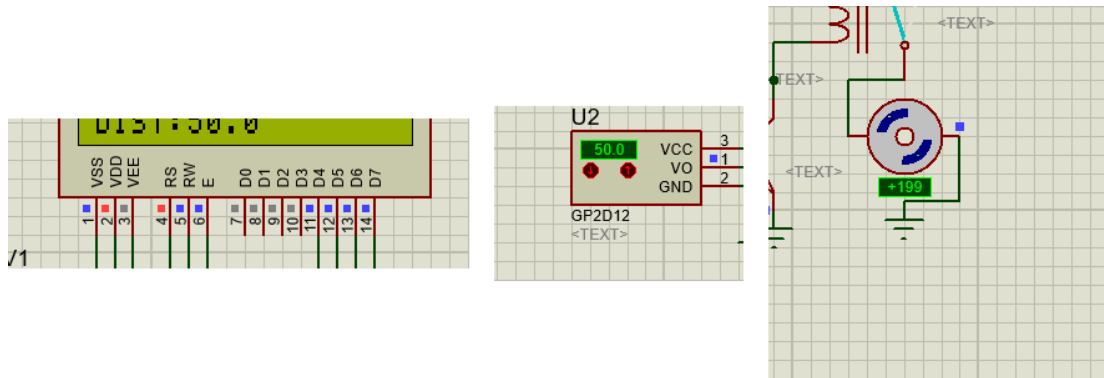


图 2 图 1 局部放大：LCD 显示、传感器读数、电机转速（左起）

图 1、图 2 中：① 上位机软件 Title 显示“23009201247 郑墨涵”，发送窗口显示已发送学号 23009201247；② 红外测距传感器（GP2D12）手动调节至 50.0cm；③ LCD 第一行显示 ID:23009201247，第二行显示 DIST:50.0；④ 上位机接收窗口实时显示接收到的距离值 DIST:50.0（每 500ms 一帧）；⑤ 50.0cm 高于阈值 37cm，直流电机转动（电机模型标签显示转速约 199，上位机状态栏显示“运转中”）。

传感器面板、LCD、上位机软件接收窗口三处显示的距离值一致，均为 50.0cm。

3. 距离低于距离阈值时的截图（30cm < 37cm，电机停止）

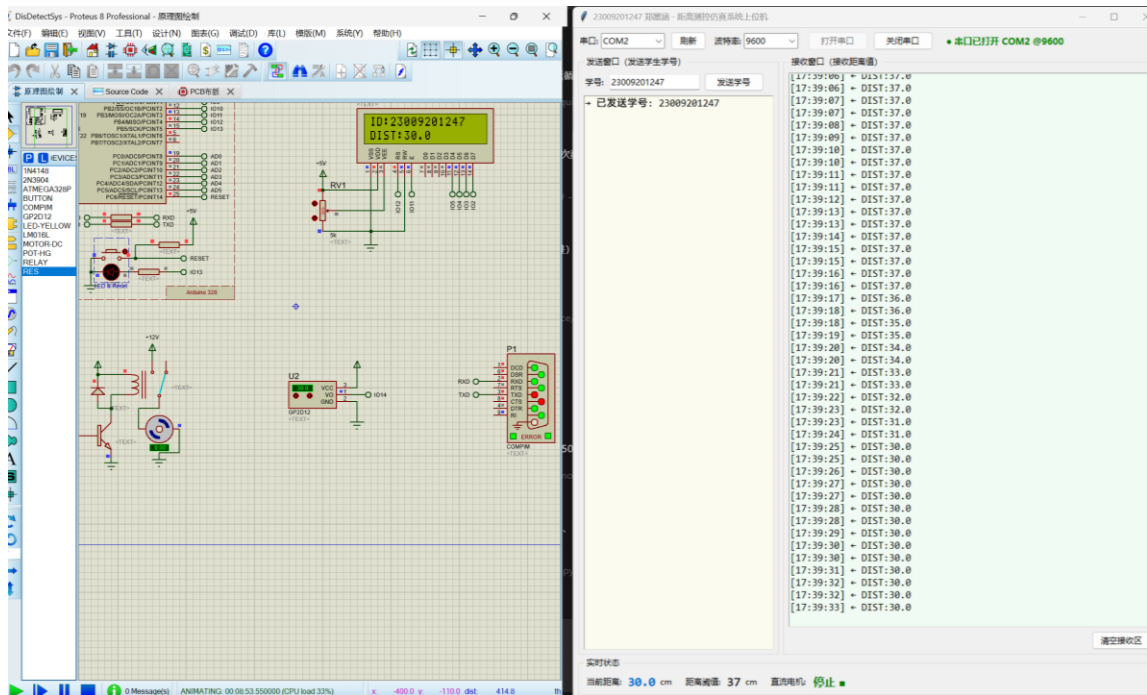


图3 距离 30.0cm 低于阈值 37cm 时系统全景（电机停止）

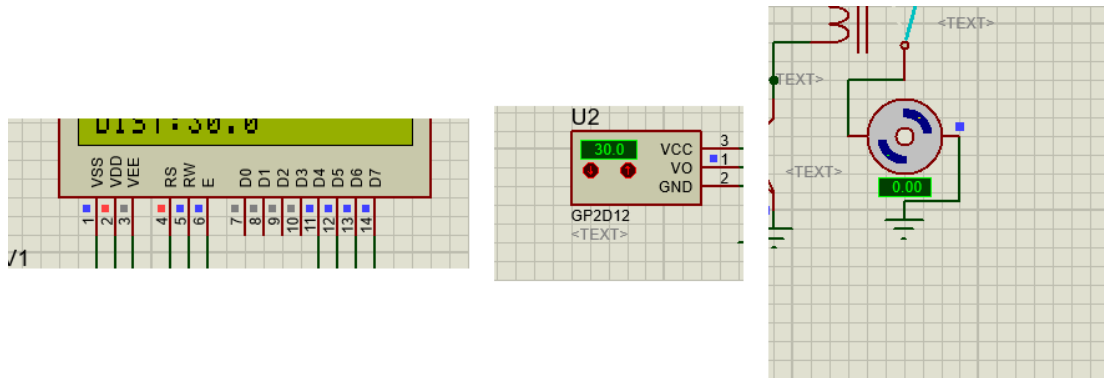


图4 图3 局部放大：LCD 显示、传感器读数、电机转速 0.00（左起）

图3、图4 中：① 发送窗口显示已发送学号 23009201247；② 传感器手动调节至 30.0cm，低于阈值 37cm；③ LCD 显示 ID:23009201247、DIST:30.0；④ 上位机接收窗口实时显示 DIST:30.0；⑤ 直流电机停止（电机标签显示转速 0.00，上位机状态栏显示“停止”）。三处距离值一致，均为 30.0cm。

4. 阈值边界验证（37cm = 阈值）

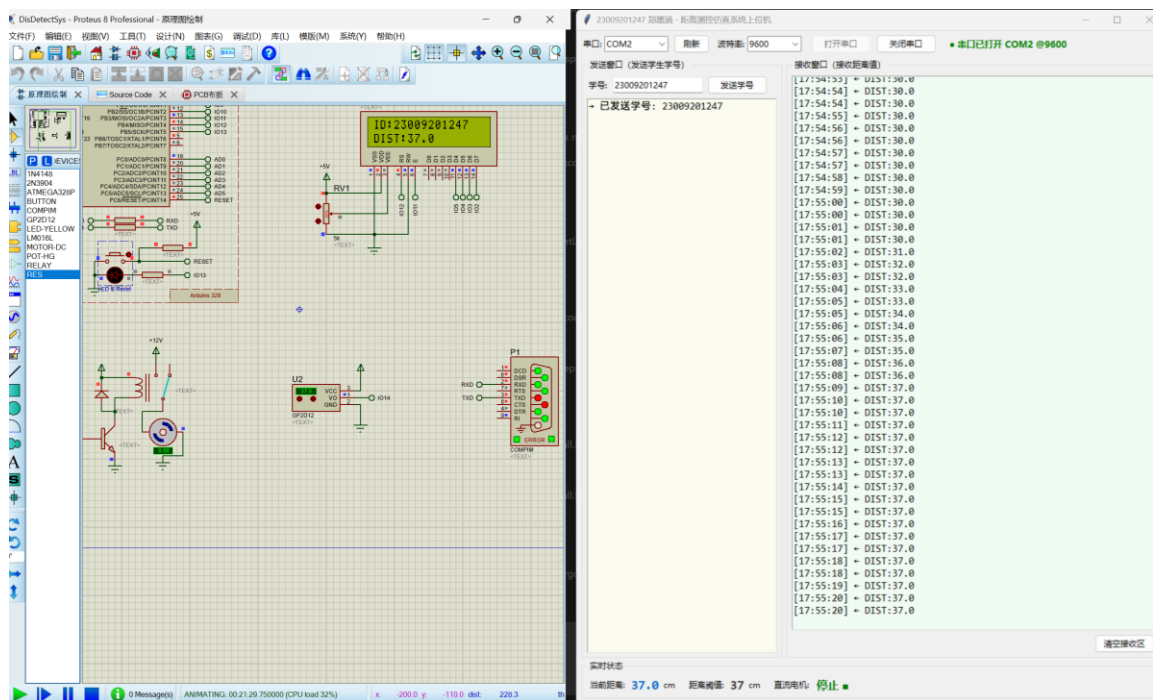


图5 距离 37.0cm 等于阈值 37cm 时电机停止（边界验证）

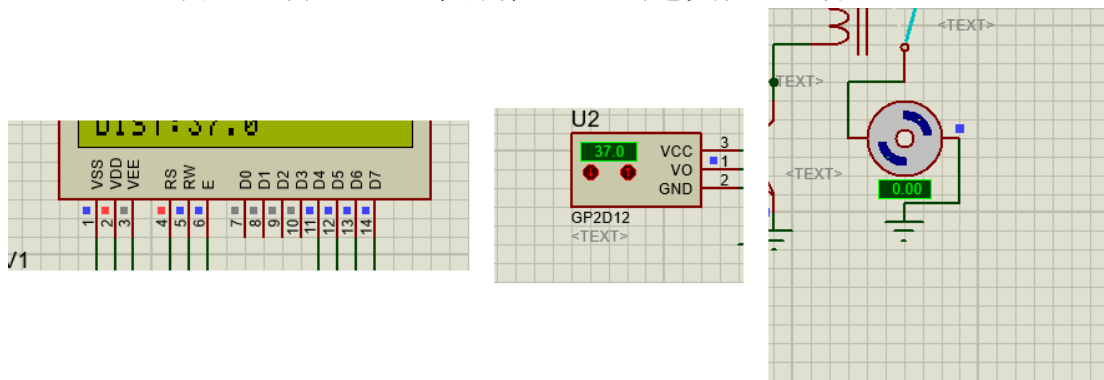


图6 图5 局部放大：LCD 显示 DIST:37.0、传感器 37.0、电机转速 0（左起）

传感器调至 37.0cm，恰好等于阈值。按“低于（含等于）阈值时电机停止”的规则，电机停止转动（转速为 0），上位机状态栏显示“停止”，验证了阈值判断的边界正确。三处距离值一致，均为 37.0cm。

综上：距离 50.0cm（高于阈值）时电机转动，37.0cm（等于阈值）和 30.0cm（低于阈值）时电机停止，系统控制逻辑与题目要求完全一致。

三、参考文献

- [1] Arduino Documentation. LiquidCrystal Library & Serial Interface[EB/OL]. <https://docs.arduino.cc>.
- [2] Labcenter Electronics. Proteus 8 Professional User Manual & VSM 器件模型库（GP2D12、COMPIM）[Z].

- [3] Sharp Corporation. GP2D12 Infrared Distance Measuring Sensor Datasheet[Z].
- [4] 张毅刚. 单片机原理及应用（第3版）[M]. 北京：高等教育出版社, 2016.
- [5] pyserial Documentation. pyserial 3.5[EB/OL]. <https://pyserial.readthedocs.io>.

四、程序设计

1.Arduino 程序源代码。程序完成：串口接收学号并自动计算阈值（30+学号末位）、LCD1602 显示学号与距离、GP2D12 距离采集与换算（64 次过采样后按特性方程反变换）、电机阈值控制、每 500ms 向上位机上传距离值。运行结果与图 1~图 6 相符。

```
/*
 * =====
 * 题目：距离测控仿真系统（A 级达标测试）
 * 平台：Arduino UNO (ATmega328P) + Proteus 8 仿真
 * 学生：郑墨涵 学号：23009201247
 * -----
 * 功能：
 * 1. 通过串口(9600bps,经 COMPIM 与虚拟串口对接)与 PC 上位机双向通信：
 *    PC 发送学号(以'\n'结尾) -> Arduino 保存并计算阈值(30+学号末位)，
 *    应答 OK:ID=xxx, 并每 500ms 向 PC 发送当前距离值 DIST:xx.x。
 * 2. LCD1602 第一行显示 ID:学号，第二行显示 DIST:距离值。
 * 3. GP2D12 红外测距传感器(手动调节)输出接 A0(IO14)。
 * 4. 距离值 高于 阈值(30+学号末位)cm 时启动直流电机(IO7->继电器)；
 *    距离值 低于(含等于) 阈值时电机停止。
 * -----
 * GP2D12 测距原理（Proteus 模型特性方程）：
 * 
$$V = 15.61824 * d^{(-0.817)} - 0.02869$$
 （输出限幅 0.39V~2.6V）
 * 反变换：
 * 
$$d = ((V + 0.02869) / 15.61824)^{(-1.224)}$$

 * =====
 */
#include <LiquidCrystal.h>

/* LCD1602: RS=12, E=11, D4=5, D5=4, D6=3, D7=2 (RW 接地) */
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

const uint8_t SENSOR_PIN = A0; /* GP2D12 VO 输出 -> A0(IO14) */
const uint8_t MOTOR_PIN = 7; /* IO7 -> 4.7k -> NPN -> 继电器->电机 */

/* GP2D12 特性常数（见文件头说明） */
const float GP_K = 15.61824f; /* 增益 K1*K2 = 15.312*1.02 */
const float GP_B = 0.02869f; /* K3 - K2*K4*TEMP = 0.07-1.02*1.5e-3*27 */
const float GP_E = -1.224f; /* 指数 -1/0.817 */
```

```

const unsigned long SEND_MS = 500; /* 距离值上传周期 */

String studentID = ""; /* 上位机发来的学号 */
bool idOK = false; /* 是否已收到学号 */
int threshold = 30; /* 距离阈值 = 30 + 学号末位数 */

char rx_buf[24]; /* 串口接收缓冲 */
uint8_t rx_len = 0;
unsigned long t_last = 0;

/* 读取距离值(cm): 64次过采样抑制ADC量化误差, 再按特性方程反变换 */
float readDistanceCM()
{
    unsigned int sum = 0;
    for (uint8_t i = 0; i < 64; i++) sum += analogRead(SENSOR_PIN);
    float v = sum * (5.0f / 64.0f / 1023.0f); /* 平均电压 */
    float d = pow((v + GP_B) / GP_K, GP_E); /* 反变换 */
    return constrain(d, 0.0f, 999.0f);
}

void setup()
{
    pinMode(MOTOR_PIN, OUTPUT);
    digitalWrite(MOTOR_PIN, LOW); /* 上电电机默认停止 */
    lcd.begin(16, 2);
    lcd.print("Wait for ID...");
    lcd.setCursor(0, 1);
    lcd.print("DIST:--.-");
    Serial.begin(9600);
}

void loop()
{
    /* ---- 1. 接收上位机发来的学号 (以回车换行结束) ---- */
    while (Serial.available()) {
        char c = Serial.read();
        if (c == '\n' || c == '\r') {
            if (rx_len > 0) {
                rx_buf[rx_len] = '\0';
                bool valid = true;
                for (uint8_t i = 0; i < rx_len; i++)
                    if (!isDigit(rx_buf[i])) { valid = false; break; }
                if (valid) {
                    studentID = String(rx_buf);
                    /* 阈值 = 30 + 学号末位数, 如末位为7则阈值为37cm */
                    int last = studentID.charAt(studentID.length() - 1) - '0';

```

```

        threshold = 30 + last;
        idOK = true;
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("ID:");
        lcd.print(studentID);          /* 第一行显示 ID:学号 */
        Serial.print("OK:ID=");        /* 应答上位机 */
        Serial.println(studentID);
        t_last = 0;                    /* 促使立即发送一帧距离 */
    }
}
rx_len = 0;
}
else if (rx_len < sizeof(rx_buf) - 1) {
    rx_buf[rx_len++] = c;
}
}

/* ---- 2. 采集距离并控制电机 ---- */
float dist = readDistanceCM();

/* 传感器以 1cm 步进设定距离，读数四舍五入到整数厘米，
   使 LCD/上位机显示值与传感器面板显示值完全一致 */
float dist_show = roundf(dist);

/* 距离高于阈值 -> 电机转动；低于(含等于)阈值 -> 电机停止 */
digitalWrite(MOTOR_PIN, (idOK && dist_show > threshold) ? HIGH : LOW);

/* ---- 3. 实时上传距离值，并刷新 LCD 第二行 ---- */
unsigned long now = millis();
if (idOK && (now - t_last >= SEND_MS)) {
    t_last = now;
    char val[12];
    dtostrf(dist_show, 4, 1, val);      /* "50.0" 整数厘米+1 位小数 */
    lcd.setCursor(0, 1);
    lcd.print("DIST:");
    lcd.print(val);
    lcd.print(" ");                    /* 清除残留字符 */
    Serial.print("DIST:");
    Serial.println(val);               /* 上位机显示距离值 */
}
}

```

2. 上位机程序源代码（Python 3 + pyserial + tkinter）。GUI Title 显示学号和姓名，含串口打开/关闭、发送窗口（发送并显示学号）、接收窗口（显示应答与实时距离值）、

实时状态栏（当前距离/阈值/电机状态）。运行结果与图 1～图 6 相符。

```
# -*- coding: utf-8 -*-
"""
=====

题目：距离测控仿真系统 —— PC 上位机软件
学生：郑墨涵    学号：23009201247
-----

功能：
    1. 串口打开/关闭功能（与 VSPD 虚拟串口、Proteus COMPIM 对接）；
    2. 发送窗口：向 Arduino 发送学号并显示已发送内容；
    3. 接收窗口：显示 Arduino 返回的应答 (OK:ID=xxx) 与实时距离值 (DIST:xx.x)；
    4. 实时显示当前距离、距离阈值 (30+学号末位) 与电机运行状态。
运行环境：Python 3.x + pyserial (pip install pyserial)
=====
"""

import serial
import serial.tools.list_ports
import threading
import queue
import datetime
import tkinter as tk
from tkinter import ttk, messagebox

STUDENT_ID = "23009201247"
STUDENT_NAME = "郑墨涵"
BAUD = 9600

class HostApp:
    def __init__(self, root: tk.Tk):
        self.root = root
        self.ser = None
        self.rx_thread = None
        self.rx_queue = queue.Queue()
        self.rx_stop = threading.Event()
        self.sent_id = ""

        root.title(f"{STUDENT_ID} {STUDENT_NAME} - 距离测控仿真系统上位机")
        root.geometry("860x600")
        root.minsize(760, 520)
        self._build_ui()
        self._refresh_ports()
        self.root.after(80, self._poll_rx_queue)

# ----- UI -----
```

```

def _build_ui(self):
    style = ttk.Style(self.root)
    try:
        style.theme_use("vista")
    except tk.TclError:
        pass

    top = ttk.Frame(self.root, padding=(8, 6))
    top.pack(side=tk.TOP, fill=tk.X)

    ttk.Label(top, text="串口:").pack(side=tk.LEFT)
    self.port_var = tk.StringVar()
    self.port_combo = ttk.Combobox(top, textvariable=self.port_var,
                                    width=10, state="readonly")
    self.port_combo.pack(side=tk.LEFT, padx=(2, 8))
    ttk.Button(top, text="刷新", width=6,
               command=self._refresh_ports).pack(side=tk.LEFT)
    ttk.Label(top, text="波特率:").pack(side=tk.LEFT, padx=(10, 0))
    self.baud_var = tk.StringVar(value=str(BAUD))
    ttk.Combobox(top, textvariable=self.baud_var, width=8,
                 values=["9600", "19200", "38400", "57600", "115200"],
                 state="readonly").pack(side=tk.LEFT, padx=(2, 10))
    self.btn_open = ttk.Button(top, text="打开串口", command=self._open_port)
    self.btn_open.pack(side=tk.LEFT, padx=4)
    self.btn_close = ttk.Button(top, text="关闭串口", command=self._close_port,
                                state=tk.DISABLED)
    self.btn_close.pack(side=tk.LEFT, padx=4)
    self.status_var = tk.StringVar(value="● 串口已关闭")
    self.status_label = tk.Label(top, textvariable=self.status_var, fg="gray",
                                 font=("Microsoft YaHei", 10, "bold"))
    self.status_label.pack(side=tk.LEFT, padx=12)

    body = ttk.Frame(self.root, padding=(8, 2))
    body.pack(fill=tk.BOTH, expand=True)

    # ---- 发送窗口 ----
    send_box = ttk.LabelFrame(body, text=" 发送窗口（发送学生学号） ", padding=6)
    send_box.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 6))
    row = ttk.Frame(send_box)
    row.pack(fill=tk.X)
    ttk.Label(row, text="学号:").pack(side=tk.LEFT)
    self.id_var = tk.StringVar(value=STUDENT_ID)
    self.id_entry = ttk.Entry(row, textvariable=self.id_var, width=18)
    self.id_entry.pack(side=tk.LEFT, padx=6)
    self.btn_send = ttk.Button(row, text="发送学号", command=self._send_id,
                               state=tk.DISABLED)
    self.btn_send.pack(side=tk.LEFT)

```

```

self.send_log = tk.Text(send_box, height=12, width=34, state=tk.DISABLED,
                        font=("Consolas", 11), bg="#FBFBF0")
self.send_log.pack(fill=tk.BOTH, expand=True, pady=(6, 0))

# ---- 接收窗口 ----
recv_box = ttk.LabelFrame(body, text=" 接收窗口（接收距离值） ", padding=6)
recv_box.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
self.recv_log = tk.Text(recv_box, height=16, state=tk.DISABLED,
                        font=("Consolas", 11), bg="#F0FBF4")
self.recv_log.pack(fill=tk.BOTH, expand=True)
recv_row = ttk.Frame(recv_box)
recv_row.pack(fill=tk.X, pady=(6, 0))
ttk.Button(recv_row, text="清空接收区",
           command=self._clear_recv).pack(side=tk.RIGHT)

# ---- 底部状态栏 ----
bottom = ttk.LabelFrame(self.root, text=" 实时状态 ", padding=(8, 4))
bottom.pack(side=tk.BOTTOM, fill=tk.X, padx=8, pady=(0, 8))
self.dist_var = tk.StringVar(value="--.-")
self.thr_var = tk.StringVar(value="--")
self.motor_var = tk.StringVar(value="--")
ttk.Label(bottom, text="当前距离:").pack(side=tk.LEFT)
tk.Label(bottom, textvariable=self.dist_var, fg="#0066CC",
        font=("Consolas", 14, "bold")).pack(side=tk.LEFT, padx=(4, 0))
ttk.Label(bottom, text="cm").pack(side=tk.LEFT, padx=(2, 16))
ttk.Label(bottom, text="距离阈值:").pack(side=tk.LEFT)
tk.Label(bottom, textvariable=self.thr_var, fg="#333333",
        font=("Consolas", 14, "bold")).pack(side=tk.LEFT, padx=(4, 0))
ttk.Label(bottom, text="cm").pack(side=tk.LEFT, padx=(2, 16))
ttk.Label(bottom, text="直流电机:").pack(side=tk.LEFT)
self.motor_label = tk.Label(bottom, textvariable=self.motor_var,
                           font=("Microsoft YaHei", 12, "bold"))
self.motor_label.pack(side=tk.LEFT, padx=6)

# ----- 串口 -----
def _refresh_ports(self):
    ports = [p.device for p in serial.tools.list_ports.comports()]
    self.port_combo["values"] = ports
    if ports and not self.port_var.get():
        self.port_var.set(ports[0])

def _open_port(self):
    port = self.port_var.get()
    if not port:
        messagebox.showwarning("提示", "请先选择串口！")
    return

```

```

try:
    self.ser = serial.Serial(port, int(self.baud_var.get()),
                             timeout=0.2)
except serial.SerialException as e:
    messagebox.showerror("串口错误", f"无法打开 {port}:\n{e}")
    self.ser = None
    return

self.rx_stop.clear()
self.rx_thread = threading.Thread(target=self._reader, daemon=True)
self.rx_thread.start()
self.btn_open["state"] = tk.DISABLED
self.btn_close["state"] = tk.NORMAL
self.btn_send["state"] = tk.NORMAL
self.status_var.set(f"● 串口已打开 {port} @{self.baud_var.get()}")
self.status_label.config(fg="green")

def _close_port(self):
    self.rx_stop.set()
    if self.rx_thread:
        self.rx_thread.join(timeout=1)
    if self.ser and self.ser.is_open:
        self.ser.close()
    self.ser = None
    self.btn_open["state"] = tk.NORMAL
    self.btn_close["state"] = tk.DISABLED
    self.btn_send["state"] = tk.DISABLED
    self.status_var.set("● 串口已关闭")
    self.status_label.config(fg="gray")

# ----- 发送/接收 -----
def _send_id(self):
    if not (self.ser and self.ser.is_open):
        messagebox.showwarning("提示", "请先打开串口!")
        return
    sid = self.id_var.get().strip()
    if not sid.isdigit():
        messagebox.showwarning("提示", "学号必须为数字!")
        return
    self.ser.write((sid + "\n").encode("ascii"))
    self.sent_id = sid
    self._log(self.send_log, f"→ 已发送学号: {sid}")
    thr = 30 + int(sid[-1])
    self.thr_var.set(str(thr))
    self._append_recv(f"[本地] 已发送学号 {sid}, 阈值={thr}cm, 等待距离数据...")

def _reader(self):

```

```

buf = b""
while not self.rx_stop.is_set():
    try:
        data = self.ser.read(128)
    except serial.SerialException:
        break
    if data:
        buf += data
        while b"\n" in buf:
            line, buf = buf.split(b"\n", 1)
            text = line.decode("ascii", "replace").strip()
            if text:
                self.rx_queue.put(text)

def _poll_rx_queue(self):
    try:
        while True:
            text = self.rx_queue.get_nowait()
            self._append_recv(f"← {text}")
            if text.startswith("DIST:"):
                try:
                    dist = float(text[5:].strip())
                    self.dist_var.set(f"{dist:.1f}")
                    if self.sent_id:
                        thr = 30 + int(self.sent_id[-1])
                        if dist > thr:
                            self.motor_var.set("运转中 ▶▶")
                            self.motor_label.config(fg="#CC2200")
                        else:
                            self.motor_var.set("停止 ■")
                            self.motor_label.config(fg="#007700")
                except ValueError:
                    pass
            except queue.Empty:
                pass
        self.root.after(80, self._poll_rx_queue)

# ----- 工具 -----
def _log(self, widget: tk.Text, msg: str):
    widget.config(state=tk.NORMAL)
    widget.insert(tk.END, msg + "\n")
    widget.see(tk.END)
    widget.config(state=tk.DISABLED)

def _append_recv(self, msg: str):
    ts = datetime.datetime.now().strftime("%H:%M:%S")

```

```

        self._log(self.recv_log, f"[{ts}] {msg}")

    def _clear_recv(self):
        self.recv_log.config(state=tk.NORMAL)
        self.recv_log.delete("1.0", tk.END)
        self.recv_log.config(state=tk.DISABLED)

def main():
    root = tk.Tk()
    HostApp(root)
    root.mainloop()

if __name__ == "__main__":
    main()

```